■ Complete System Documentation

Table of Contents

System Overview

Purpose

The Pizza Store Hygiene Violation Detection System is designed to monitor workers in real-time to ensure they comply with hygiene protocols, specifically requiring the use of a scooper when handling ingredients from designated containers.

Key Capabilities

• **Real-time Video Analysis**: Processes video streams at configurable frame rates
• **Object Detection**: Identifies hands, persons, pizzas, and scoopers using YOLO v12
• **Hand Tracking**: Tracks individual hands across frames with state management
• **Violation Detection**: Detects when hands enter ROI without a scooper
• **Data Logging**: Stores violations in SQLite database with frame exports
• **Web Interface**: Modern UI for real-time monitoring and statistics

System Requirements

• **Python**: 3.8 or higher

• **RAM**: 4GB minimum, 8GB recommended

• **Storage**: 500MB for code + dependencies, additional for video files

• **CPU**: Multi-core recommended for better performance

• **GPU**: Optional but recommended for faster YOLO inference

• **Kafka**: Required only for full microservices mode

Architecture Details

Microservices Components

1. Frame Reader Service

• **Purpose**: Reads video frames and publishes to Kafka

• **Input**: Video file or camera stream

• **Output**: Base64-encoded frames to Kafka topic `video-frames`

• **Configuration**: Frame rate, video path

2. Detection Service

• **Purpose**: Core detection and violation logic

• **Input**: Frames from Kafka

• **Processing**:

• YOLO object detection

• Hand tracking and state management

• ROI intersection detection

• Scooper presence detection

• Violation logic evaluation

• **Output**: Annotated frames and detection results to Kafka topic `detections`

3. Streaming Service

• **Purpose**: Web API and video streaming

• **Components**:

• REST API endpoints

• WebSocket server

• MJPEG video stream

• **Output**: HTTP/WebSocket responses to frontend

4. Kafka Message Broker

• **Purpose**: Asynchronous communication between services

• **Topics**:

• `video-frames`: Raw video frames

• `detections`: Detection results and annotated frames

5. Frontend UI

• **Purpose**: User interface for monitoring

• **Features**:

• Live video stream display

• Real-time statistics

• Violation alerts

• Detection information

Data Flow

```
Video → Frame Reader → Kafka → Detection Service → Kafka → Streaming Service → Frontend
                                                    ↓
                                                SQLite Database
```

Installation Guide

Step 1: Environment Setup

```
Create virtual environment
python -m venv venv

Activate (Windows)
venv\Scripts\activate

Activate (Linux/Mac)
source venv/bin/activate
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Key dependencies:

• `ultralytics` - YOLO v12

• `opencv-python` - Video processing

• `fastapi` - Web framework

• `uvicorn` - ASGI server

• `kafka-python` - Kafka client (for microservices mode)

• `websockets` - WebSocket support

Step 3: Kafka Setup (Optional)

For full microservices mode:

1. Download Apache Kafka

2. Extract to `C:\kafka` (or your preferred location)

3. Start Zookeeper:

`bash

cd C:\kafka

bin\windows\zookeeper-server-start.bat config\zookeeper.properties

`

4. Start Kafka (in new terminal):

`bash

cd C:\kafka

bin\windows\kafka-server-start.bat config\server.properties

`

Step 4: Verify Installation

```
python check_setup.py
```

This will verify:

• Python version

• Required packages

• File structure

• Kafka connection (if enabled)

Configuration Guide

Video Configuration

Edit `config/settings.py`:

```
VIDEO_PATH = r'C:\path\to\video.mp4'
FRAME_RATE = 5  # Frames per second to process
VIDEO_WIDTH = 1280
VIDEO_HEIGHT = 720
```

Model Configuration

```
MODEL_PATH = r'C:\path\to\yolo12m-v2 (1).pt'
CONFIDENCE_THRESHOLD = 0.4  # YOLO detection confidence
IOU_THRESHOLD = 0.5  # Intersection over Union threshold
```

ROI Configuration

```
DEFAULT_ROIS = [
    {
        "name": "Protein Container",
        "coords": [400, 250, 515, 590],  # [x1, y1, x2, y2]
        "color": (0, 255, 0)  # Green color for visualization
    }
]
```

How to set ROI coordinates:

1. Run the system and view the video
2. Identify the protein container location
3. Note the pixel coordinates (x1, y1) for top-left corner
4. Note the pixel coordinates (x2, y2) for bottom-right corner
5. Update `coords` in settings

Detection Parameters

```
Temporal consistency
MIN_CONSECUTIVE_FRAMES = 3  # Hand must be in ROI for N frames

Spatial requirements
OVERLAP_RATIO_THRESHOLD = 0.25  # Minimum IoU (25% of hand must overlap ROI)
MIN_OVERLAP_AREA = 1500  # Minimum pixel area overlap

Scooper detection
SCOOPER_IOU_THRESHOLD = 0.2  # IoU between hand and scooper

Violation prevention
VIOLATION_COOLDOWN_FRAMES = 20  # Frames to wait between violations
```

Kafka Configuration

```
KAFKA_BOOTSTRAP_SERVERS = 'localhost:9092'
KAFKA_TOPIC_FRAMES = 'video-frames'
KAFKA_TOPIC_DETECTIONS = 'detections'
KAFKA_GROUP_ID = 'detection-service'
```

Usage Instructions

Quick Start (Simplified Mode)

The easiest way to run the system without Kafka:

```
python run_web_demo.py
```

This will:

1. Load the YOLO model

2. Start processing the video

3. Launch web server on port 3000

4. Open browser automatically

Full Microservices Mode

1. **Start Kafka** (if not already running):

• Zookeeper on port 2181

• Kafka on port 9092

2. **Start Services** (in separate terminals):

`bash

Terminal 1: Frame Reader

python services/frame_reader/frame_reader.py

Terminal 2: Detection Service

python services/detection/detection_service.py

Terminal 3: Streaming Service

python services/streaming/streaming_service.py

`

3. **Access Web Interface**:

• Open browser to `http://localhost:3000`

Web Interface Usage

1. **Video Stream**: View live video with detection annotations
2. **Statistics Panel**: See total violations and active detections
3. **Current Detections**: Real-time object counts (hands, pizzas, scoopers)
4. **Violations List**: History of detected violations with timestamps
5. **Controls**:

• Refresh Stream: Reload video stream

• Clear Violations: Reset violation count

API Reference

REST Endpoints

GET /api/status

Get system status.

Response:

```
{
  "status": "running",
  "video_loaded": true,
  "model_loaded": true
}
```

GET /api/violations

Get violation statistics.

Response:

```
{
  "total_violations": 5,
  "last_violation": "2025-01-24T19:30:00"
}
```

GET /api/violations/list

List recent violations.

Query Parameters:

• `limit` (optional): Number of violations to return (default: 10)

Response:

```
{
  "violations": [
    {
      "id": 1,
      "timestamp": "2025-01-24T19:30:00",
      "frame_number": 150,
      "roi_name": "Protein Container",
      "violation_type": "Hand in ROI without scooper"
    }
  ]
}
```

DELETE /api/violations/clear

Clear all violations from database.

Response:

```
{
  "message": "All violations cleared",
  "deleted_count": 5
}
```

GET /stream/video

Get MJPEG video stream.

Response: MJPEG stream (multipart/x-mixed-replace)

WebSocket API

Endpoint: ws://localhost:3000/ws/detections

Connection:

```
const ws = new WebSocket('ws://localhost:3000/ws/detections');
```

Message Format:

```
{
  "type": "detection",
  "data": {
    "total_violations": 5,
    "detections": {
      "hands": [...],
      "pizzas": [...],
      "scoopers": [...]
    },
    "violations": [...]
  }
}
```

Detection Logic

Hand Tracking

Each detected hand is assigned a unique ID and tracked across frames:

```
hand_tracking_history = {
    "hand_id_1": {
        "consecutive_frames": 5,
        "last_seen": frame_number,
        "violation_triggered": False
    }
}
```

ROI Intersection

A hand is considered "in ROI" when:

1. **Overlap Ratio** ≥ 25%: At least 25% of hand bounding box overlaps ROI
2. **Center in ROI**: Hand center point is inside ROI boundaries
3. **Minimum Area** ≥ 1500 pixels: Significant overlap area

Scooper Detection

A scooper is considered "with hand" when:

• IoU (Intersection over Union) between hand and scooper > 0.2

• Scooper bounding box overlaps with hand bounding box

Violation Trigger

A violation is triggered when:

1. Hand is in ROI (meets all spatial requirements)
2. Hand remains in ROI for ≥3 consecutive frames (temporal consistency)
3. No scooper detected with the hand
4. Cooldown period has passed (prevents duplicate violations)

Violation Prevention

• **Cooldown Period**: 20 frames between violations for same hand

• **State Tracking**: Each hand tracks if it already triggered a violation

• **Temporal Consistency**: Requires multiple frames to prevent false positives

Troubleshooting

Issue: Video Not Displaying

Symptoms:

• Black screen in web interface

• "Video stream loading..." message persists

Solutions:

1. Wait 30-40 seconds for YOLO model to load

2. Check if video file exists and path is correct

3. Verify video format is supported (MP4, AVI, etc.)

4. Check console for error messages

5. Try refreshing the stream

Issue: No Violations Detected

Symptoms:

• System running but no violations logged

• Hands visible in video but not detected

Solutions:

1. **Adjust ROI Coordinates**:

• Ensure ROI box is precisely around protein container

• Not too large (includes worker) or too small (misses hands)

2. **Lower Detection Thresholds**:

`python

OVERLAP_RATIO_THRESHOLD = 0.15 # Lower from 0.25

MIN_OVERLAP_AREA = 1000 # Lower from 1500

MIN_CONSECUTIVE_FRAMES = 2 # Lower from 3

`

3. **Check Console Output**:

• Look for "Hand in container: YES" messages

• Verify hands are being detected

• Check if scooper detection is working

4. **Verify Video Content**:

• Ensure video actually contains violations

• Check if hands are clearly visible

Issue: Too Many False Positives

Symptoms:

• Violations detected when workers are using scoopers

• Violations for cleaning actions

Solutions:

1. **Increase Thresholds**:

`python

OVERLAP_RATIO_THRESHOLD = 0.35 # Increase from 0.25

MIN_OVERLAP_AREA = 2000 # Increase from 1500

MIN_CONSECUTIVE_FRAMES = 5 # Increase from 3

`

2. **Adjust Scooper Detection**:

`python

SCOOPER_IOU_THRESHOLD = 0.15 # Lower to detect scoopers more easily

`

3. **Fine-tune ROI**:

• Make ROI smaller to focus on container only

• Exclude areas where cleaning occurs

Issue: High CPU Usage

Symptoms:

• System slows down computer

• High CPU percentage in Task Manager

Solutions:

1. **Reduce Frame Rate**:

`python

FRAME_RATE = 2 # Lower from 5

`

2. **Use GPU** (if available):

• Install CUDA-enabled PyTorch

• YOLO will automatically use GPU

3. **Lower Video Resolution**:

• Resize video before processing

• Or process at lower resolution

Issue: Kafka Connection Errors

Symptoms:

• "Cannot connect to Kafka"

• "NoBrokersAvailable"

Solutions:

1. Verify Kafka is running:

`bash

Check if Kafka is listening on port 9092

netstat -an | findstr 9092

`

2. Start Kafka services:

• Zookeeper on port 2181

• Kafka on port 9092

3. Check Kafka configuration in `config/settings.py`

4. Use simplified mode (`run_web_demo.py`) which doesn't require Kafka

Performance Optimization

Frame Rate Tuning

Balance between accuracy and performance:

• **High Accuracy**: 5-10 FPS (more processing, better detection)

• **Balanced**: 3-5 FPS (recommended)

• **Performance**: 1-2 FPS (faster, may miss quick violations)

GPU Acceleration

If GPU is available:

1. Install CUDA-enabled PyTorch:

`bash

pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118

`

2. YOLO will automatically detect and use GPU

3. Expect 5-10x speedup in detection

Memory Optimization

For large videos or long-running sessions:

1. **Limit Frame Queue Size**:

`python

frame_queue = Queue(maxsize=10) # Lower from 30

`

2. **Clear Tracking History**:

• System automatically cleans up old tracking data

• Adjust `TRACKING_HISTORY_FRAMES` if needed

3. **Database Cleanup**:

• Periodically clear old violations

• Export and archive violation frames

Development Guide

Code Structure

```
services/
■■■ frame_reader/
■   ■■■ frame_reader.py      # Video reading and Kafka publishing
■■■ detection/
■   ■■■ detection_service.py # YOLO detection and violation logic
■■■ streaming/
    ■■■ streaming_service.py # FastAPI server and WebSocket

utils/
■■■ database.py              # SQLite operations
■■■ roi_manager.py           # ROI management
■■■ tracker.py               # Object tracking utilities
```

```
config/
■■■ settings.py              # Global configuration
```

Adding New Features

1. **New Detection Class**:

• Update YOLO model to include new class

• Add class to `CLASSES` dictionary in `config/settings.py`

• Update detection logic in `detection_service.py`

2. **New ROI**:

• Add ROI to `DEFAULT_ROIS` in `config/settings.py`

• Update frontend to display multiple ROIs

3. **New API Endpoint**:

• Add route in `streaming_service.py`

• Update frontend to call new endpoint

Testing

1. **Unit Tests**:

`bash

python -m pytest tests/

`

2. **Integration Tests**:

• Test with sample video

• Verify detection accuracy

• Check database logging

3. **Performance Tests**:

• Measure FPS

• Monitor memory usage

• Test with different video resolutions

Debugging

Enable debug logging:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

View detailed console output:

• Hand detection status

• ROI intersection results

• Scooper detection

• Violation triggers

Additional Resources

• **YOLO Documentation**: https://docs.ultralytics.com/

• **FastAPI Documentation**: https://fastapi.tiangolo.com/

• **Kafka Documentation**: https://kafka.apache.org/documentation/

• **OpenCV Documentation**: https://docs.opencv.org/

Last Updated: January 2025

Version: 1.0.0